

Python 2- Cost of Equity for Non-Listed Firms

Rithika Reddy Shetpalliwar

Approach

Step 1	Load & Inspect Data	Read CRSP (monthly returns, 2015–2020) and Compustat (fundamentals, FY2020). Checked variable types and ran descriptive statistics with sanity checks.
Step 2	Filter Firms	Kept firms with at least 30 monthly return observations. It is needed for a statistically reliable OLS beta estimate.
Step 3	Estimate Equity Betas	Ran OLS separately for each firm. The slope β is the levered equity beta as it reflects both business and financial risk.
Step 4	Un-lever Betas	Applied the Hamada equation to eliminate each firm's financial leverage. Merged with Compustat D/E ratios first.
Step 5	Industry Averages	Grouped firms by 3-digit SIC code. Discarded industries with fewer than 5 firms. Computed the mean un-levered beta per qualifying industry.
Step 6	Estimate Re (CAPM)	Matched each non-listed firm to its industry beta. Re-levered using the firm's own D/E ratio, then applied Re (CAPM) formula.

```
In [87]: # Import required python libraries
```

```
import pandas as pd
import statsmodels.api as sm
```

```
In [ ]: # Read CRSP Data into pandas DataFrame
```

```
crsp = pd.read_csv('crsp.csv', usecols=['PERMNO', 'date', 'CUSIP', 'RET', 'ewretd'])
```

```
In [89]: # Display the first and last few rows of crsp data
```

```
print('==== First few rows of CRSP Data ====')
print(crsp.head())
```

```
print('==== Last few rows of CRSP Data ====')
print(crsp.tail())
```

```
==== First few rows of CRSP Data ====
```

	PERMNO	date	CUSIP	RET	ewretd
0	10001	2015-01-30	36720410	-0.115245	-0.019073
1	10001	2015-02-27	36720410	0.035897	0.054120
2	10001	2015-03-31	36720410	-0.000495	-0.005112
3	10001	2015-04-30	36720410	0.011044	0.015127
4	10001	2015-05-29	36720410	-0.006951	0.003970

```
==== Last few rows of CRSP Data ====
```

	PERMNO	date	CUSIP	RET	ewretd
527106	93436	2020-08-31	88160R10	0.741452	0.038884
527107	93436	2020-09-30	88160R10	-0.139087	-0.027356
527108	93436	2020-10-30	88160R10	-0.095499	0.000584
527109	93436	2020-11-30	88160R10	0.462736	0.174412
527110	93436	2020-12-31	88160R10	0.243252	0.072853

```
In [ ]: # Read Compustat Data into pandas DataFrame
```

```
compustat = pd.read_csv('compustat.csv', usecols=['costat', 'curcd', 'datafmt', 'indfmt', 'consol', 'gvkey', 'datadate', ''])
```

```
In [91]: # Display the first and last few rows of compustat data
```

```
print('==== First few rows of Compustat Data ====')
print(compustat.head())
```

```
print('==== Last few rows of Compustat Data ====')
print(compustat.tail())
```

==== First few rows of Compustat Data ====

	costat	curcd	datafmt	indfmt	consol	gvkey	datadate	cusip	fyr	sic	\
0	I	USD	STD	INDL	C	1019	2020-12-31	001038108	12	7380	
1	A	USD	STD	INDL	C	1045	2020-12-31	02376R102	12	4512	
2	A	USD	STD	INDL	C	1050	2020-12-31	125141101	12	3564	
3	A	USD	STD	INDL	C	1076	2020-12-31	74319R101	12	6141	
4	A	USD	STD	INDL	C	1078	2020-12-31	002824100	12	3845	

	fyear	csho	dt	prcc_f
0	2020	0.162	8.668	174.00
1	2020	621.480	41021.000	15.77
2	2020	35.367	91.540	6.96
3	2020	67.723	79.516	53.87
4	2020	1771.230	19677.000	109.49

==== Last few rows of Compustat Data ====

	costat	curcd	datafmt	indfmt	consol	gvkey	datadate	cusip	fyr	\
4260	A	USD	STD	INDL	C	332115	2020-12-31	04216R102	12	
4261	I	USD	STD	INDL	C	345556	2020-12-31	30315R107	12	
4262	A	USD	STD	INDL	C	345920	2020-12-31	44888K407	12	
4263	A	USD	STD	INDL	C	345980	2020-12-31	21078F109	12	
4264	A	USD	STD	INDL	C	348892	2020-12-31	583435409	12	

	sic	fyear	csho	dt	prcc_f
4260	2836	2020	18.688	12.428	2.9849
4261	2836	2020	9.100	3.161	9.8700
4262	3524	2020	33.500	20.057	52.5800
4263	5961	2020	587.000	52.000	18.2400
4264	3550	2020	7.987	0.180	1.1300

```
In [92]: # Check type of variables in crsp and compustat dataframes
print('==== CRSP Data Types ====')
print(crsp.dtypes)

print('==== Compustat Data Types ====')
print(compustat.dtypes)
```

```

==== CRSP Data Types ====
PERMNO      int64
date        object
CUSIP       object
RET         float64
ewretd      float64
dtype: object
==== Compustat Data Types ====
costat      object
curcd       object
datafmt     object
indfmt      object
consol      object
gvkey       int64
datadate    object
cusip       object
fyr         int64
sic         int64
fyear       int64
csho        float64
dt          float64
prcc_f      float64
dtype: object

```

```

In [93]: # Descriptive statistics of CRSP and Compustat data
print('==== CRSP Data Descriptive Statistics ====')
print(crsp.describe(include='all'))

# Storing the mean and standard deviation of RET for the sanity check below.
ret_mean = crsp['RET'].mean()
ret_std  = crsp['RET'].std()

# Sanity check: monthly returns should have a mean well below 5% and a standard deviation well below 50%.
if abs(ret_mean) < 0.05 and ret_std < 0.50:
    print("\n CRSP Check Passed")
else:
    print("\n CRSP Check Failed – investigate return units or bad data")

print(f"Mean RET = {ret_mean:.6f}")
print(f"Std RET  = {ret_std:.6f}")

```

```
print('==== Compustat Data Descriptive Statistics ====')
print(compustat.describe(include='all'))

# Confirm that all firms have positive share counts and prices. Negative or zero
# values would indicate data errors and would cause the D/E ratio to be meaningless.
if (compustat['csho'] > 0).all() and (compustat['prcc_f'] > 0).all():
    print("\n COMPUSTAT Check Passed")
else:
    print("\n COMPUSTAT Check Failed – negative shares or prices found")
```

==== CRSP Data Descriptive Statistics ====

	PERMNO	date	CUSIP	RET	ewretd
count	527111.000000	527111	527111	527111.000000	527111.000000
unique	NaN	72	10625	NaN	NaN
top	NaN	2020-12-31	88160R10	NaN	NaN
freq	NaN	7824	72	NaN	NaN
mean	51510.188928	NaN	NaN	0.008498	0.008610
std	34973.998294	NaN	NaN	0.162959	0.053401
min	10001.000000	NaN	NaN	-0.993600	-0.207503
25%	15267.000000	NaN	NaN	-0.043272	-0.011301
50%	54973.000000	NaN	NaN	0.004662	0.007326
75%	88860.000000	NaN	NaN	0.049993	0.038572
max	93436.000000	NaN	NaN	19.883589	0.174412

CRSP Check Passed

Mean RET = 0.008498

Std RET = 0.162959

==== Compustat Data Descriptive Statistics ====

	costat	curcd	datafmt	indfmt	consol	gvkey	datadate \
count	4265	4265	4265	4265	4265	4265.000000	4265
unique	2	1	1	1	1	NaN	1
top	A	USD	STD	INDL	C	NaN	2020-12-31
freq	3319	4265	4265	4265	4265	NaN	4265
mean	NaN	NaN	NaN	NaN	NaN	74799.912544	NaN
std	NaN	NaN	NaN	NaN	NaN	74778.557741	NaN
min	NaN	NaN	NaN	NaN	NaN	1019.000000	NaN
25%	NaN	NaN	NaN	NaN	NaN	21833.000000	NaN
50%	NaN	NaN	NaN	NaN	NaN	35153.000000	NaN
75%	NaN	NaN	NaN	NaN	NaN	141278.000000	NaN
max	NaN	NaN	NaN	NaN	NaN	348892.000000	NaN

	cusip	fyr	sic	fyear	csho	dt \
count	4265	4265.0	4265.000000	4265.0	4265.000000	4.265000e+03
unique	4265	NaN	NaN	NaN	NaN	NaN
top	001038108	NaN	NaN	NaN	NaN	NaN
freq	1	NaN	NaN	NaN	NaN	NaN
mean	NaN	12.0	4735.093552	2020.0	257.440491	4.428682e+03
std	NaN	0.0	2289.947506	0.0	2313.960132	7.310699e+04
min	NaN	12.0	100.000000	2020.0	0.001000	0.000000e+00
25%	NaN	12.0	2836.000000	2020.0	25.890000	3.286000e+00
50%	NaN	12.0	3845.000000	2020.0	56.379000	9.579400e+01

75%	NaN	12.0	6798.000000	2020.0	144.507000	1.197305e+03
max	NaN	12.0	9997.000000	2020.0	143840.848000	3.923563e+06

	prcc_f
count	4265.000000
unique	NaN
top	NaN
freq	NaN
mean	48.086384
std	307.479088
min	0.000100
25%	3.760000
50%	13.230000
75%	40.280000
max	17800.000000

COMPUSTAT Check Passed

```
In [94]: # From the crsp dataframe that has the stock market data, keep only those firms (CUSIPS) that have at least 30 mont
crsp_filtered = crsp.groupby('CUSIP').filter(lambda x: x['RET'].count() >= 30)

print("Original shape:", crsp.shape)
print("Filtered shape:", crsp_filtered.shape)
```

Original shape: (527111, 5)

Filtered shape: (484611, 5)

```
In [95]: # ESTIMATE MARKET MODEL (OLS) for each company (CUSIP) to get the beta of each
```

```
def estimate_beta(group):
    """
    Runs OLS regression:
    RET = alpha + beta * ewretd
    """
    X = group['ewretd']
    y = group['RET']

    X = sm.add_constant(X) # add intercept

    model = sm.OLS(y, X, missing='drop').fit()
```



```
return model.params
```

```
In [96]: # Apply regression per CUSIP
params = crsp_filtered.groupby('CUSIP').apply(estimate_beta)

# Reset index for clean dataframe
params = params.reset_index()

# Rename beta column
params = params.rename(columns={'ewretd': 'beta'})

print(params)
```

	CUSIP	const	beta
0	00030710	-0.049431	2.808096
1	00032010	-0.021440	1.877168
2	00036020	0.013597	0.569260
3	00036110	-0.001239	1.631806
4	00037520	0.001655	0.890797
...	...	...	...
7662	Y8564M10	-0.017436	1.273191
7663	Y8565J10	-0.038058	2.453739
7664	Y8897Y23	-0.153231	0.595666
7665	Y9384M10	-0.006291	1.356183
7666	Y9530810	0.013617	0.152276

[7667 rows x 3 columns]

```
/var/folders/5x/c7r80s1d2g3gbz8cvrtr46c00000gn/T/ipykernel_76921/258229037.py:2: FutureWarning: DataFrameGroupBy.apply operated on the grouping columns. This behavior is deprecated, and in a future version of pandas the grouping columns will be excluded from the operation. Either pass `include_groups=False` to exclude the groupings or explicitly select the grouping columns after groupby to silence this warning.
params = crsp_filtered.groupby('CUSIP').apply(estimate_beta)
```

```
In [97]: # PROCESS COMPUSTAT DATA

# Extract first 8 digits of CUSIP
compustat['CUSIP'] = compustat['cusip'].astype(str).str[:8]

# Create SIC 3-digit
```

```

compustat['sic3'] = compustat['sic'] // 10

# Calculate Market Equity = PRCC_F * CSH0
compustat['ME'] = compustat['prcc_f'] * compustat['csho']

# Calculate D/E ratio
compustat['DE'] = compustat['dt'] / compustat['ME']

# Drop extra columns that we will not be using
comp_clean = compustat[['CUSIP', 'sic3', 'DE']]

print(comp_clean.head())

```

	CUSIP	sic3	DE
0	00103810	738	0.307507
1	02376R10	451	4.185500
2	12514110	356	0.371881
3	74319R10	614	0.021796
4	00282410	384	0.101463

In [98]: # MERGE BETAS WITH LEVERAGE DATA

```

merged = pd.merge(params, comp_clean, on='CUSIP', how='inner')

print(merged.head())

```

	CUSIP	const	beta	sic3	DE
0	00037520	0.001655	0.890797	361	0.122732
1	00081T10	-0.006884	1.928036	399	1.526297
2	00089910	-0.016467	1.180520	283	0.477457
3	00090Q10	-0.009481	1.750378	738	1.486984
4	00105510	0.002120	0.830197	632	0.256516

In [99]: # UNLEVER BETA

```

# Formula: beta_unlevered = beta / (1 + (1 - tax_rate)*DE)
tax_rate = 0.30

merged['beta0'] = merged['beta'] / (1 + (1 - tax_rate) * merged['DE'])

print(merged.head())

```

	CUSIP	const	beta	sic3	DE	beta0
0	00037520	0.001655	0.890797	361	0.122732	0.820321
1	00081T10	-0.006884	1.928036	399	1.526297	0.932135
2	00089910	-0.016467	1.180520	283	0.477457	0.884802
3	00090010	-0.009481	1.750378	738	1.486984	0.857655
4	00105510	0.002120	0.830197	632	0.256516	0.703819

```
In [100... # Check how many firms are in each sic3 code
industry_counts = merged.groupby('sic3').size()
print(industry_counts)
```

```
sic3
10      3
100     12
104     32
109      3
122      6
..
871      3
872      1
873     10
874     16
999      8
Length: 224, dtype: int64
```

```
In [101... # FILTER INDUSTRIES WITH >= 5 FIRMS
valid_industries = industry_counts[industry_counts >= 5].index
merged_filtered = merged[merged['sic3'].isin(valid_industries)]

print(f"\n Industries with >= 5 firms: {len(valid_industries)}")
print(f" Number of firms retained: {len(merged_filtered)}")
```

```
Industries with >= 5 firms: 104
Number of firms retained: 2432
```

```
In [102... # CALCULATE INDUSTRY AVERAGE UNLEVERED BETAS
ibetas = merged_filtered.groupby('sic3')['beta0'].mean().reset_index()

print(ibetas.head())
```

	sic3	beta0
0	100	1.157969
1	104	0.938144
2	122	0.683350
3	131	1.281697
4	138	1.275762

```
In [ ]: # READ NON-LISTED FIRMS DATA
nl = pd.read_csv('CFA2024nonlist.csv',usecols=['Firm','sic3','D/E'] )

print(nl.head())
```

	Firm	sic3	D/E
0	A	357	0.08
1	B	357	0.12
2	C	104	0.20
3	D	371	0.31
4	E	109	0.35

```
In [104... # MERGE INDUSTRY BETAS WITH NON-LISTED FIRMS
nl_merged = pd.merge(nl, ibetas, on='sic3', how='inner')

print(nl_merged.head())
```

	Firm	sic3	D/E	beta0
0	A	357	0.080	0.950066
1	B	357	0.120	0.950066
2	C	104	0.200	0.938144
3	D	371	0.310	1.295802
4	F	371	0.452	1.295802

```
In [105... # CALCULATE COST OF EQUITY (Re)
# METHOD: 1) LEVER UP BETA + CAPM

rf = 0.045      # risk-free rate
mrp = 0.08      # market risk premium

# Lever beta
nl_merged['beta_levered'] = nl_merged['beta0'] * (1 + (1 - tax_rate) * nl_merged['D/E'])

# CAPM
```

```
nl_merged['Re'] = rf + nl_merged['beta_levered'] * mrp

print(nl_merged[['sic3', 'D/E', 'beta0', 'beta_levered', 'Re']])
```

	sic3	D/E	beta0	beta_levered	Re
0	357	0.080	0.950066	1.003270	0.125262
1	357	0.120	0.950066	1.029872	0.127390
2	104	0.200	0.938144	1.069484	0.130559
3	371	0.310	1.295802	1.576992	0.171159
4	371	0.452	1.295802	1.705794	0.181464
5	100	0.700	1.157969	1.725374	0.183030
6	122	0.750	0.683350	1.042109	0.128369
7	731	0.760	0.933075	1.429471	0.159358
8	208	0.780	0.689826	1.066471	0.130318
9	208	0.910	0.689826	1.129245	0.135340
10	208	1.050	0.689826	1.196848	0.140748
11	122	1.200	0.683350	1.257364	0.145589
12	494	1.200	0.085725	0.157734	0.057619
13	357	1.340	0.950066	1.841228	0.192298
14	495	1.650	0.634614	1.367594	0.154407
15	499	1.700	0.402445	0.881354	0.115508
16	283	1.450	1.192872	2.403638	0.237291
17	366	0.640	0.883168	1.278827	0.147306
18	371	0.230	1.295802	1.504427	0.165354
19	451	0.970	0.629086	1.056235	0.129499
20	581	0.430	0.740753	0.963720	0.122098
21	596	0.130	0.948546	1.034863	0.127789
22	731	0.810	0.933075	1.462129	0.161970